```
%%writefile relu_2stage_bf16.cu
#include <cuda_bf16.h>

////////////////////////////////////////////////////////////
// BFLOAT16 ReLU Activation (2 logical stages)
// Each thread processes 4 × __nv_bfloat16 elements
////////////////////////////////////////////////////////////

__global__ void relu_bf16(const __nv_bfloat16* input,
                          __nv_bfloat16* output,
                          int N)
{
    int idx  = blockIdx.x * blockDim.x + threadIdx.x;
    int base = idx * 4;

    if (base + 3 < N)
    {
        ////////////////////////////////////////////////////
        // Stage 1 : Load and Convert to FP32
        ////////////////////////////////////////////////////
        float x0 = __bfloat162float(input[base + 0]);
        float x1 = __bfloat162float(input[base + 1]);
        float x2 = __bfloat162float(input[base + 2]);
        float x3 = __bfloat162float(input[base + 3]);

        ////////////////////////////////////////////////////
        // Stage 2 : ReLU + Convert back to BF16 + Store
        ////////////////////////////////////////////////////
        x0 = (x0 > 0.0f) ? x0 : 0.0f;
        x1 = (x1 > 0.0f) ? x1 : 0.0f;
        x2 = (x2 > 0.0f) ? x2 : 0.0f;
        x3 = (x3 > 0.0f) ? x3 : 0.0f;

        output[base + 0] = __float2bfloat16(x0);
        output[base + 1] = __float2bfloat16(x1);
        output[base + 2] = __float2bfloat16(x2);
        output[base + 3] = __float2bfloat16(x3);
    }
}
```

```
Writing relu_2stage_bf16.cu
```

```
!nvcc -ptx -O0 -Xptxas -O0 relu_2stage_bf16.cu -o relu_2stage_bf16.ptx
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed i
```

```
!nvcc -ptx relu_2stage_bf16.cu -o relu_2stage_bf16.ptx -arch=sm_80
```

```
from google.colab import files
files.download("relu_2stage_bf16.ptx")
```

```
!nvcc -ptx relu_2stage_bf16.cu -o relu_2stage_bf16.ptx -arch=sm_80
```

```
%%writefile vecMul_2stage_bf16.cu
#include <cuda_bf16.h>

////////////////////////////////////////////////////////////
// BFLOAT16 VECTOR MULTIPLY (2 logical stages)
// Each thread processes 4 __nv_bfloat16 elements
////////////////////////////////////////////////////////////

__global__ void vecMul_bf16(const __nv_bfloat16* A,
                            const __nv_bfloat16* B,
                            __nv_bfloat16* C,
                            int N)
{
    int idx  = blockIdx.x * blockDim.x + threadIdx.x;
    int base = idx * 4;

    if (base + 3 < N)
    {
        ////////////////////////////////////////////////////
        // Stage 1 : Load and Convert to FP32
        ////////////////////////////////////////////////////
        float a0 = __bfloat162float(A[base + 0]);
        float b0 = __bfloat162float(B[base + 0]);
```

```
        float b0 = __bfloat162float(B[base + 0]);

        float a1 = __bfloat162float(A[base + 1]);
        float b1 = __bfloat162float(B[base + 1]);

        float a2 = __bfloat162float(A[base + 2]);
        float b2 = __bfloat162float(B[base + 2]);

        float a3 = __bfloat162float(A[base + 3]);
        float b3 = __bfloat162float(B[base + 3]);

        ////////////////////////////////////////////////////////
        // Stage 2 : Multiply + Convert back to BF16 + Store
        ////////////////////////////////////////////////////////
        C[base + 0] = __float2bfloat16(a0 * b0);
        C[base + 1] = __float2bfloat16(a1 * b1);
        C[base + 2] = __float2bfloat16(a2 * b2);
        C[base + 3] = __float2bfloat16(a3 * b3);
    }
}
```

```
Writing vecMul_2stage_bf16.cu
```

```
%%writefile vecFMA_2stage_bf16.cu
#include <cuda_bf16.h>

////////////////////////////////////////////////////////////
// BFLOAT16 VECTOR FMA (2 logical stages)
// Each thread processes 4 × __nv_bfloat16 elements
////////////////////////////////////////////////////////////

__global__ void vecFMA_bf16(const __nv_bfloat16* A,
                            const __nv_bfloat16* B,
                            const __nv_bfloat16* C,
                            __nv_bfloat16* D,
                            int N)
{
    int idx  = blockIdx.x * blockDim.x + threadIdx.x;
    int base = idx * 4;

    if (base + 3 < N)
    {
        ////////////////////////////////////////////////////////
        // Stage 1 : Load and Convert to FP32
        ////////////////////////////////////////////////////////
        float a0 = __bfloat162float(A[base + 0]);
        float b0 = __bfloat162float(B[base + 0]);
        float c0 = __bfloat162float(C[base + 0]);

        float a1 = __bfloat162float(A[base + 1]);
        float b1 = __bfloat162float(B[base + 1]);
        float c1 = __bfloat162float(C[base + 1]);

        float a2 = __bfloat162float(A[base + 2]);
        float b2 = __bfloat162float(B[base + 2]);
        float c2 = __bfloat162float(C[base + 2]);

        float a3 = __bfloat162float(A[base + 3]);
        float b3 = __bfloat162float(B[base + 3]);
        float c3 = __bfloat162float(C[base + 3]);

        ////////////////////////////////////////////////////////
        // Stage 2 : FMA + Convert back to BF16 + Store
        ////////////////////////////////////////////////////////
        D[base + 0] = __float2bfloat16(a0 * b0 + c0);
        D[base + 1] = __float2bfloat16(a1 * b1 + c1);
        D[base + 2] = __float2bfloat16(a2 * b2 + c2);
        D[base + 3] = __float2bfloat16(a3 * b3 + c3);
    }
}
```

```
Writing vecFMA_2stage_bf16.cu
```

```
%%writefile vecSub_2stage_bf16.cu
#include <cuda_bf16.h>

////////////////////////////////////////////////////////////
// BFLOAT16 VECTOR SUBTRACTION (2 logical stages)
// Each thread processes 4 __nv_bfloat16 elements
////////////////////////////////////////////////////////////

__global__ void vecSub_bf16(const __nv_bfloat16* A,
```

```
                           const __nv_bfloat16* B,
                           __nv_bfloat16* C,
                           int N)
{
    int idx  = blockIdx.x * blockDim.x + threadIdx.x;
    int base = idx * 4;

    if (base + 3 < N)
    {
        ////////////////////////////////////////////////////////
        // Stage 1 : Load and Convert to FP32
        ////////////////////////////////////////////////////////
        float a0 = __bfloat162float(A[base + 0]);
        float b0 = __bfloat162float(B[base + 0]);

        float a1 = __bfloat162float(A[base + 1]);
        float b1 = __bfloat162float(B[base + 1]);

        float a2 = __bfloat162float(A[base + 2]);
        float b2 = __bfloat162float(B[base + 2]);

        float a3 = __bfloat162float(A[base + 3]);
        float b3 = __bfloat162float(B[base + 3]);

        ////////////////////////////////////////////////////////
        // Stage 2 : Subtract + Convert back to BF16 + Store
        ////////////////////////////////////////////////////////
        C[base + 0] = __float2bfloat16(a0 - b0);
        C[base + 1] = __float2bfloat16(a1 - b1);
        C[base + 2] = __float2bfloat16(a2 - b2);
        C[base + 3] = __float2bfloat16(a3 - b3);
    }
}
```

```
Writing vecSub_2stage_bf16.cu
```

```
%%writefile vecAdd_2stage_bf16.cu
#include <cuda_bf16.h>

////////////////////////////////////////////////////////////
// BFLOAT16 VECTOR ADDITION (2 logical stages)
// Each thread processes 4 __nv_bfloat16 elements
////////////////////////////////////////////////////////////

__global__ void vecAdd_bf16(const __nv_bfloat16* A,
                           const __nv_bfloat16* B,
                           __nv_bfloat16* C,
                           int N)
{
    int idx  = blockIdx.x * blockDim.x + threadIdx.x;
    int base = idx * 4;

    if (base + 3 < N)
    {
        ////////////////////////////////////////////////////////
        // Stage 1 : Load and Convert to FP32
        ////////////////////////////////////////////////////////
        float a0 = __bfloat162float(A[base + 0]);
        float b0 = __bfloat162float(B[base + 0]);

        float a1 = __bfloat162float(A[base + 1]);
        float b1 = __bfloat162float(B[base + 1]);

        float a2 = __bfloat162float(A[base + 2]);
        float b2 = __bfloat162float(B[base + 2]);

        float a3 = __bfloat162float(A[base + 3]);
        float b3 = __bfloat162float(B[base + 3]);

        ////////////////////////////////////////////////////////
        // Stage 2 : Add and Convert back to BF16 + Store
        ////////////////////////////////////////////////////////
        C[base + 0] = __float2bfloat16(a0 + b0);
        C[base + 1] = __float2bfloat16(a1 + b1);
        C[base + 2] = __float2bfloat16(a2 + b2);
        C[base + 3] = __float2bfloat16(a3 + b3);
    }
}
```

```
Writing vecAdd_2stage_bf16.cu
```

```
!nvcc -ptx vecAdd_2stage_bf16.cu  -o vecAdd_2stage_bf16.ptx  -arch=sm_80
```

```
!nvcc -ptx vecSub_2stage_bf16.cu  -o vecSub_2stage_bf16.ptx  -arch=sm_80
!nvcc -ptx vecMul_2stage_bf16.cu  -o vecMul_2stage_bf16.ptx  -arch=sm_80
!nvcc -ptx vecFMA_2stage_bf16.cu  -o vecFMA_2stage_bf16.ptx  -arch=sm_80
```

```
!ls -lh *.ptx
```

```
-rw-r--r-- 1 root root 2.1K Feb 28 07:58 relu_2stage_bf16.ptx
-rw-r--r-- 1 root root 2.7K Feb 28 08:20 vecAdd_2stage_bf16.ptx
-rw-r--r-- 1 root root 3.4K Feb 28 08:20 vecFMA_2stage_bf16.ptx
-rw-r--r-- 1 root root 2.7K Feb 28 08:20 vecMul_2stage_bf16.ptx
-rw-r--r-- 1 root root 2.7K Feb 28 08:20 vecSub_2stage_bf16.ptx
```

```
!zip bf16_ptx_files.zip *.ptx
```

```
  adding: relu_2stage_bf16.ptx (deflated 68%)
  adding: vecAdd_2stage_bf16.ptx (deflated 73%)
  adding: vecFMA_2stage_bf16.ptx (deflated 76%)
  adding: vecMul_2stage_bf16.ptx (deflated 73%)
  adding: vecSub_2stage_bf16.ptx (deflated 73%)
```

```
from google.colab import files
files.download("bf16_ptx_files.zip")
```